

CaseWhy — Technical Brief for USCIS Production API Access

Prepared for USCIS Torch Developer Support ahead of the Case Status API production-access demo.

Applicant: CaseWhy LLC (Florida), sole founder and operator: Peter.

What CaseWhy is: a free web application that lets people track the status of their own USCIS case by receipt number, explains what a status means in plain language, and shows published processing times and Visa Bulletin data. It is not a law firm and gives no legal advice. Live at app.casewhy.com.

USCIS Demo Criteria

#	Criterion	Status	Where it's demonstrated
1	Data entry usability	Pass	Receipt-number field with real-time format feedback, error copy, sign-in, Spanish toggle, mobile-responsive layout.
2	Application data converts to a JSON payload	Pass	A "Show technical details" panel on the on-demand check button renders the exact outbound request (method, URL, headers, redacted token) generated by the same code path that runs the call.
3	OAuth 2.0 authentication	Pass	Client Credentials grant, token cached and refreshed server-side, credentials never reach the browser.
4	HTTPS response handling, success and error, message shown in the UI	Pass	Both success and error responses render in the interface; USCIS's own error text is shown directly, not a generic substitute.
5	Case status tracking	Pass	Add a case, dashboard display, on-demand and scheduled checks, status-history table, change-notification email.

A `demo_id` header, assigned by USCIS, is sent on every Case Status API request during the demo window (`src/lib/uscis/client.ts, demoIdHeader()`).

File upload / encoded-document handling is a FOIA API criterion, not a Case Status API one, and is out of scope for this demo.

1. Data entry usability

The receipt-number field gives real-time format feedback as the user types (3 letters followed by 10 digits), with clear error copy, without ever blocking submission — a well-formed number still might not correspond to a real case. **[1]** The app supports English and Spanish throughout, is fully responsive on mobile, and a new user can sign up and track their first case in under two minutes.

2. JSON payload conversion

`getCaseStatus()` builds the outbound request to USCIS's Case Status API from the entered receipt number. **[2]** The exact request that will be sent — method, URL, and headers, with the access token redacted — is visible in the dashboard via a "Show technical details" toggle on the on-demand check button, generated by the same function that makes the real call, not a separate description. **[3]**

3. OAuth 2.0 authentication

CaseWhy uses the OAuth 2.0 Client Credentials grant. **[4]** `getAccessToken()` requests a token from USCIS's token endpoint using the client ID and secret, and caches it in memory, refreshing automatically shortly before expiry rather than on every call. **[5]** Sandbox and production use separate credential pairs and separate token/API base URLs, selected by a single environment variable — there is no code path where they can be mixed up. **[6]** Credentials are read from environment variables, never committed to source control, and live only as encrypted environment variables in Vercel.

The USCIS API client runs exclusively on the server. **Client ID, client secret, and access tokens never reach the browser** — the browser only ever talks to CaseWhy's own server, which in turn talks to USCIS. **[7]**

4. HTTPS response handling — success and error

Successful responses are parsed against a typed shape covering both the standard response and the IOE-prefix variant (whose date fields are sometimes absent), and rendered in the dashboard. **[8]**

Every non-2xx response is caught and its message shown directly in the interface. A malformed receipt number returns a 422 with USCIS's own explanation of the format requirement; an unknown but well-formed receipt number returns a 404 with USCIS's own message that the case isn't recognized. **[9]** `extractUscisErrorMessage()` reads the message from either a flat `{"message": "..."} body or an errors[] array, so it degrades gracefully regardless of which shape a given response uses. [10]`

There is no automatic retry logic — a failed call is reported as a failure for that specific check, not retried in a loop. Receipt numbers are never written to logs in plaintext.

5. Case status tracking

A tracked case stores an encrypted receipt number, case type, and last known status; `case_status_history` stores one row per USCIS history entry (service-center prefix only, never the full receipt number). [11] The scheduled status-check job compares the freshly fetched status against the last known one and — only on a genuine change, never on the first check — sends a Postmark email and, if enabled, a push notification. [12]

Adding a case is a short signed-in flow; removing one is immediate and self-serve (`untrackCase()`, a real, ownership-checked delete). [13] Free accounts can track up to 3 cases; the paid tier raises this in tiers up to a hard ceiling. [14]

Additional Technical & Operational Detail

Architecture and hosting

CaseWhy is a Next.js 15 application (App Router) hosted on Vercel, with PostgreSQL (Neon) for data storage via Drizzle ORM. [15] Transactional email runs through Postmark; billing (for the optional paid tier) through Stripe. Every server route runs on Vercel's standard Node.js runtime, chosen specifically because the app's database access needs a real TCP connection pool, which the Edge runtime doesn't support. [16] There is no native mobile app; CaseWhy is an installable Progressive Web App.

File upload (document vault)

Not a Case Status API criterion — a FOIA API criterion, out of scope for this demo, included here for completeness. The paid tier includes a working document-upload feature: PDF, JPEG, or PNG up to 10 MB, stored in Vercel Blob under private access (never a public URL), with the original filename encrypted before storage. [17] Downloads are proxied through CaseWhy's own server rather than exposing a direct Blob URL. [18] No uploaded document is ever sent to USCIS or any third party.

Rate limiting and volume

Nothing in the API client throttles calls directly — pacing comes from how CaseWhy schedules them. The production status-check job runs on a fixed schedule per tracked case; an on-demand check is a paid-tier feature naturally rate-limited by real human clicks. Volume scales linearly and predictably with the number of tracked cases, not with traffic spikes. A separate example of the team's general rate-limiting practice: the anonymous "ask a question" feature enforces a durable, database-backed lifetime cap per IP address. [19]

Privacy, security, and data handling

Receipt numbers are treated as personally identifiable information: encrypted with AES-256-GCM before being written to the database, with the encryption key held separately from the database itself. [20] No CaseWhy staff member can view a receipt number in plaintext through any administrative tool.

What is sent to the AI service (Claude, via Anthropic, through the Vercel AI Gateway): form type, processing center, dates, status text/description, full status history, and matched policy background. **The raw receipt number is never included**, by design in the prompt-construction code. [21] The one exception: the paid-tier escalation-letter drafting tool includes the receipt number in the drafted letter itself, since the letter has to identify the case to USCIS or a congressional office.

Authentication is via Neon Auth, supporting both password and magic-link sign-in. [22] Admin access is gated by a single allow-listed email, fail-closed if unset. [23]

Removing an individual tracked case is immediate and self-serve. Full account deletion is a request to `privacy@casewhy.com`, completed within 30 days. [24] Users are required to actively acknowledge a material change to the Privacy Policy or Terms of Use the next time they sign in, not just notified by email. [25]

HTTPS/TLS is provided by Vercel's platform. CaseWhy has not yet layered custom security headers (CSP, HSTS) on top of the platform defaults, and does not run a formal automated dependency-vulnerability scan beyond normal npm tooling.

Business and operations

A free tier (3 tracked cases, 3 AI questions/month, full access to all reference data) and a paid "Plus" tier (\$9.99/month, \$22.99/quarter, or \$66.99/year). [26] **Nothing from the USCIS Case Status API is ever pay-walled** — status checking is free at every tier; Plus adds more tracked cases, unlimited AI questions, on-demand checks, the document vault, and drafting tools. Billing runs on Stripe, currently in test mode — CaseWhy has processed no real production payments to date. [27]

Support runs through 13 real, monitored email aliases on the domain, including `security@`, `privacy@`, `legal@`, and `abuse@`. [28] If production API access were ever revoked, CaseWhy has no scraping fallback and would show an honest degraded-service message rather than attempt an unauthorized alternative path to USCIS data.

Sandbox testing

CaseWhy ran a dedicated 5-business-day sandbox volume test, September 14–18, 2026, using the app's real production code path, not a mock. [29] Each run drew from a confirmed pool of real sandbox test receipt numbers spanning 7 form types, at roughly 50 calls per run (a mix of successful and deliberate-error lookups), 18 runs/day across the sandbox's 7 AM–8 PM ET operating hours — roughly 900 calls/day, under the sandbox's published 1,000-request/day, 5-transactions-per-second ceiling. The test window completed with no quota-exceeded responses and no unexpected errors.

[1] `src/lib/uscis/receipt-number.ts`; `src/app/dashboard/ReceiptNumberInput.tsx`.

[2] `src/lib/uscis/client.ts`, `getCaseStatus()`.

[3] `src/lib/uscis/client.ts`, `describeCaseStatusRequest()`; `src/app/dashboard/CheckNowButton.tsx`.

[4] `src/lib/uscis/client.ts`, `getAccessToken()`.

[5] `src/lib/uscis/client.ts`, in-memory token cache keyed by client ID.

[6] `src/lib/config.ts`, `getUscisConfig()`, `USCIS_ENV`.

[7] `src/lib/uscis/client.ts`; `src/middleware.ts` (runtime: "nodejs").

[8] `src/lib/uscis/client.ts`, `RawCaseStatus` type.

[9] Confirmed live against the sandbox for both a malformed (422) and unknown (404) receipt number.

[10] `src/lib/uscis/client.ts`, `extractUscisErrorMessage()`.

[11] `src/lib/db/schema.ts`, `trackedCases` and `caseStatusHistory` tables.

[12] `src/lib/uscis/check-status.ts`, `checkTrackedCaseNow()`.

[13] src/app/dashboard/actions.ts, trackCase(), untrackCase().

[14] src/lib/billing/tier.ts, TIER_LIMITS, PLUS_HARD_CEILING_MAX_CASES.

[15] package.json; src/lib/db/client.ts.

[16] src/middleware.ts; next.config.ts (experimental.nodeMiddleware).

[17] src/app/api/documents/route.ts; src/lib/db/schema.ts, caseDocuments table.

[18] src/app/api/documents/[id]/route.ts.

[19] src/lib/get-help/anonymous-usage.ts, LIFETIME_CAP_PER_IP.

[20] src/lib/db/crypto.ts.

[21] src/lib/ai/case-context.ts, buildCaseContext().

[22] src/lib/auth/server.ts; src/app/auth/sign-in/page.tsx.

[23] src/lib/auth/admin.ts, isAdminEmail().

[24] Privacy Policy, Section 7, www.casewhy.com/privacy.html.

[25] src/lib/policy/versions.ts; the acknowledgment-gate middleware.

[26] src/lib/billing/pricing.ts, getAllEffectivePrices().

[27] src/lib/stripe/client.ts.

[28] scripts/seed-email-alias-configs.ts; email_alias_configs table.

[29] src/app/api/cron/uscis-volume-test/route.ts; src/lib/uscis/sandbox-volume-test.ts.